المدرسة الوطنية العليا للذكاء الاصطناعي وعلوم المعطيات - تارودانت
ÉCOLE NATIONALE SUPÉRIEURE DE L'INTELLIGENCE ARTIFICIELLE
ET SCIENCES DES DONNÉES - TAROUDANT

Taroudant Heritage Shield

Rapport d'Audit de Sécurité

Analyse Comparative — Correction des Vulnérabilités

Équipe Blue Team

BENZEROUAL Fatima Ezzahra
IKRAM boudrifi

Tâche principale

Chiffrement des rapports d'expertise
Données sensibles sur la fragilité des structures publiques

Avril 2026

Table des matières

1 Résumé Exécutif **3**

1.1 App with AI — État initial 3

1.2 App without AI — État initial 4

1.3 Recommandations prioritaires 4

2 Introduction **5**

2.1 Présentation du système 5

2.2 Contexte de l'exercice 5

2.3 Objectifs de l'audit 5

3 Périmètre et Méthodologie **6**

3.1 Dans le périmètre 6

3.2 Méthodologie 6

3.3 Normes de référence 7

4 App with AI — Vulnérabilités et Corrections **8**

4.1 F-01 — Secret JWT Codé en Dur 8

4.2 F-02 — Rapports Experts Stockés en Clair 9

4.3 F-03 — Absence d'Authentification 2FA 9

4.4 F-04 — Absence de Limitation de Débit 10

4.5 F-05 — Politique CORS Sans Restriction 10

4.6 Synthèse des corrections F-06 à F-10 11

5 App without AI — Vulnérabilités et Corrections **12**

5.1 V-01 — Clé AES Codée en Dur dans SQL 12

5.2 V-02 — Clé AES Codée en Dur dans reports.py 13
5.3 V-03 — Stored XSS dans le Champ Notes 13
5.4 V-04 — Absence de Rate Limiting 14
5.5 Synthèse des corrections V-05 à V-07 14

6 Architecture du Chiffrement — Tâche Principale 15

6.1 Architecture implémentée 15
6.2 Faille critique détectée 16

7 Évaluation des Risques 17

7.1 App with AI — Scores CVSS 17
7.2 App without AI — Synthèse 17

8 Comparaison des Deux Applications 18

8.1 Tableau comparatif 18
8.2 Analyse qualitative 18
8.3 Conclusion générale 19

9 Annexes 20

9.1 Annexe A — Scripts de Migration SQL 20
9.2 Annexe B — Variables d'Environnement 20
9.3 Annexe C — Glossaire 21

1 Résumé Exécutif

Un audit de sécurité complet a été mené sur la plateforme **Taroudant Heritage Shield**, une application dédiée à la surveillance de l'intégrité structurelle des monuments historiques de la région de Taroudant. L'audit couvre deux versions distinctes : une première version générée automatiquement par IA (*App with AI*) et une seconde version développée sans assistance IA (*App without AI*).

Conclusion principale : Les deux versions présentaient un **risque critique** avant correction.
L'App with AI affichait un spectre plus large de vulnérabilités (10), tandis que l'App without AI
présentait des failles plus graves sur le chiffrement des données sensibles.

1.1 App with AI — État initial

La plateforme présentait un état de **RISQUE ÉLEVÉ** avant remédiation, avec un score de risque composite de **73/100**. Dix vulnérabilités ont été identifiées, dont deux de niveau **CRITIQUE**.

ID	Constat	Sévérité	Statut
F-01	Secret JWT codé en dur	CRITIQUE	✓Corrigé
F-02	Rapports experts stockés en clair	CRITIQUE	✓Corrigé
F-03	Absence d'authentification 2FA	ÉLEVÉ	✓Corrigé
F-04	Aucune limitation de débit login	ÉLEVÉ	✓Corrigé
F-05	Politique CORS sans restriction	ÉLEVÉ	✓Corrigé
F-06	TTL JWT = 7 jours	MOYEN	✓Corrigé
F-07	`autocommit=True`	MOYEN	✓Corrigé
F-08	`debug=True` en production	MOYEN	✓Corrigé
F-09	Absence de journaux d'audit	MOYEN	✓Corrigé
F-10	Mot de passe base de données vide	FAIBLE	✓Corrigé

1.2 App without AI — État initial

L'application développée sans IA présentait également des vulnérabilités critiques, principalement centrées sur l'implémentation du chiffrement et la configuration des secrets.

ID	Constat	Sévérité	Statut
V-01	Clé AES codée en dur dans fichier SQL	CRITIQUE	✓ Corrigé
V-02	Clé AES codée en dur dans `reports.py`	CRITIQUE	✓ Corrigé
V-03	Stored XSS dans champ notes	ÉLEVÉ	✓ Corrigé
V-04	Absence de rate limiting sur login	ÉLEVÉ	✓ Corrigé
V-05	Exposition erreur MySQL brute	ÉLEVÉ	✓ Corrigé
V-06	Procédures stockées absentes	ÉLEVÉ	✓ Corrigé
V-07	Secrets JWT placeholder dans `.env`	CRITIQUE	✓ Corrigé

1.3 Recommandations prioritaires

1. **Regénérer immédiatement** tous les secrets et les placer dans des variables d'environnement
2. **Exécuter le script de migration** pour chiffrer les données existantes
3. **Activer la 2FA** sur les comptes Autorité et Admin
4. **Déployer les fichiers corrigés** avec gestion d'erreur générique
5. **Mettre en place un rate limiter** sur l'authentification

2 Introduction

2.1 Présentation du système

La plateforme **Taroudant Heritage Shield** sert les autorités gouvernementales, les ingénieurs en structures et les experts du patrimoine pour évaluer et surveiller les monuments historiques de Taroudant (remparts saadiens, portes historiques, mosquées, kasbahs).

Données sensibles traitées
Diagnostics structurels détaillés, analyses de fissures, recommandations de remédiation
La divulgation pourrait révéler des vulnérabilités exploitables dans l'infrastructure publique.

2.2 Contexte de l'exercice

Cet exercice met en jeu deux équipes complémentaires :

Red Team (Attaque)	Blue Team (Défense)
BENSBAA M. BENTOUMI Oumaima	BENZEROUAL Fatima Ez-zahra IKRAM boudrifi

2.3 Objectifs de l'audit

- Identifier les vulnérabilités de sécurité dans la couche d'authentification et d'autorisation
- Évaluer le risque lié au stockage de rapports sensibles sans chiffrement
- Mesurer la résistance de l'application aux attaques par force brute et au détournement de session
- Fournir du code de remédiation fonctionnel et directement déployable
- **Tâche principale : Garantir le chiffrement des rapports d'expertise sur la fragilité structurelle**

3 Périmètre et Méthodologie

3.1 Dans le périmètre

App with AI

- `app.py` — Routes Flask
- `db.py` — Gestion connexions
- `schema.sql` — Tables
- `procedures.sql` — Procédures
- `triggers.sql` — Déclencheurs
- Scripts de migration

App without AI

- Backend FastAPI
- Base MySQL 8.0
- RBAC (4 niveaux)
- `auth.py`, `main.py`
- `reports.py`, `inspections.py`
- `config.py`

3.2 Méthodologie

L'audit suit une méthodologie d'analyse structurée, alignée sur **OWASP Top 10 (2021)** et le **CWE/SANS Top 25**.

1. **Phase 1 — Reconnaissance** : Lecture complète des fichiers sources
2. **Phase 2 — Modélisation des Menaces** : Application du modèle STRIDE
3. **Phase 3 — Identification des Vulnérabilités** : Revue de code manuelle
4. **Phase 4 — Scoring des Risques** : CVSS v3.1
5. **Phase 5 — Remédiation** : Correctifs prêts pour la production

3.3 Normes de référence

Norme	Description
OWASP Top 10 (2021)	A02 Défaillances cryptographiques, A07 Défaillances d'authentification
CWE-798	Utilisation de données d'identification codées en dur
CWE-312	Stockage en clair d'informations sensibles
CWE-307	Restriction insuffisante des tentatives d'authentification
NIST SP 800-38D	Recommandation du mode AES-GCM
RFC 6238	TOTP — Mot de passe à usage unique basé sur le temps

4 App with AI — Vulnérabilités et Corrections

4.1 F-01 — Secret JWT Codé en Dur

CRITIQUE — CVSS v3.1 : 9,8

Composant : `app.py` — `create_app()` — ligne 36

Avant correction

```
app.config["JWT_SECRET"] = os.getenv("JWT_SECRET",
    "dev-change-me")
# Si JWT_SECRET n'est pas defini, "dev-change-me" est utilise.
# Tout attaquant connaissant cette chaine peut forger des jetons
    valides.
```

Après correction

```
_jwt_secret = os.getenv("JWT_SECRET", "")
if not _jwt_secret or _jwt_secret == "dev-change-me":
    if os.getenv("FLASK_ENV") == "production":
        sys.exit(1)  # Arret force -- refus de demarrer
    _jwt_secret = secrets.token_hex(32)
app.config["JWT_SECRET"] = _jwt_secret
```

Impact

Un attaquant peut forger un jeton JWT pour n'importe quel utilisateur et rôle (Admin inclus) sans mot de passe.

4.2 F-02 — Rapports Experts Stockés en Clair

CRITIQUE — CVSS v3.1 : 9,1

Composant : `schema.sql`, `app.py` — `reports_create()`

Avant correction

```
cur.execute(
    'INSERT INTO RAPPORT (..., diagnostic_structurel, ...) VALUES
   (..., %s, ...)',
    (diagnostic, ...)  # Stockage en texte brut
)
```

Après correction

```
# encryption.py -- AES-256-GCM
cur.execute('INSERT INTO RAPPORT ... VALUES (%s, ...)',
    (encrypt_field(diagnostic), ...))
```

Impact

Une injection SQL ou une fuite de sauvegarde donne accès direct aux données sensibles.

4.3 F-03 — Absence d'Authentification 2FA

ÉLEVÉ — CVSS v3.1 : 8,1

Correction

Implémentation TOTP (RFC 6238) avec émission d'un jeton de pré-authentification de 5 minutes.

4.4 F-04 — Absence de Limitation de Débit

ÉLEVÉ — CVSS v3.1 : 7,5

Correction

```
ip = request.remote_addr or 'unknown'
if not default_limiter.check('login', ip):
    return jsonify({'error': 'Trop de tentatives. Reessayez dans 60s.'}), 429
# 5 tentatives / 60 secondes par IP
```

4.5 F-05 — Politique CORS Sans Restriction

ÉLEVÉ — CVSS v3.1 : 7,4

Correction

```
CORS(app, origins=[o.strip() for o in os.getenv('CORS_ALLOWED_ORIGINS',
    'http://localhost:3000').split(',')], supports_credentials=True)
```

4.6 Synthèse des corrections F-06 à F-10

ID	Constat	Sévérité	Correction
F-06	TTL JWT = 7 jours	MOYEN	TTL réduit à 60 minutes
F-07	`autocommit=True`	MOYEN	`autocommit=False` + commit explicite
F-08	`debug=True` en production	MOYEN	`debug` dépend de `FLASK_ENV`
F-09	Absence de journalisation	MOYEN	Table `AUDIT_LOG` + AnomalyDetector
F-10	Mot de passe BDD vide	FAIBLE	Vérification production + `exit(1)`

5 App without AI — Vulnérabilités et Corrections

5.1 V-01 — Clé AES Codée en Dur dans SQL

CRITIQUE — Clé exposée dans le code source

Fichier : `02_stored_procedures.sql`

Avant correction

```sql
-- FAILLE : cle visible dans le code source
SET v_encrypted_content = AES_ENCRYPT(
    v_report_content,
    SHA2('TaroudantHeritage2026Key', 256)  -- <- EN DUR
);
```

Après correction

```sql
CREATE PROCEDURE GenerateMonumentReport (
    IN p_monument_id INT,
    IN p_inspection_id INT,
    IN p_generated_by INT,
    IN p_aes_key BLOB  -- Nouveau parametre depuis Python
)
BEGIN
    SET v_encrypted_content = AES_ENCRYPT(
        v_report_content,
        SHA2(p_aes_key, 256)  -- Jamais ecrit dans le SQL
    );
END
```

5.2 V-02 — Clé AES Codée en Dur dans reports.py

CRITIQUE

Après correction

```
def _get_aes_key() -> str:
    key = settings.AES_REPORT_KEY
    if not key:
        raise RuntimeError("AES_REPORT_KEY non definie dans
    .env.")
    return key
```

5.3 V-03 — Stored XSS dans le Champ Notes

ÉLEVÉ

Preuve Red Team

```
curl -X POST http://192.168.1.182:8000/api/inspections/ \
    -d '{"notes": "<script>alert(XSS)</script>"}'
# Reponse : {"inspection_id":4} <- Script ACCEPTE
```

Correction

```
def sanitize_text(value: Optional[str]) -> Optional[str]:
    if value is None:
        return None
    cleaned = re.sub(r'<[^>]+>', '', value)
    cleaned = html.escape(cleaned, quote=True)
    return cleaned.strip()
```

5.4 V-04 — Absence de Rate Limiting

ÉLEVÉ

Correction

```
from slowapi import Limiter
limiter = Limiter(key_func=get_remote_address,
    default_limits=["200/minute"])

@router.post('/login')
@limiter.limit("10/minute")  # 10 tentatives max / min / IP
async def login(...):
    ...
```

Résultat

À partir de la 11e tentative par minute, le serveur répond **HTTP 429 Too Many Requests**.

5.5 Synthèse des corrections V-05 à V-07

ID	Constat	Sévérité	Correction
V-05	Exposition erreur MySQL brute	ÉLEVÉ	Handler erreur générique + migration SQL
V-06	Procédures stockées absentes	ÉLEVÉ	Exécution scripts dans l'ordre
V-07	Secrets JWT placeholder	CRITIQUE	Génération secrets aléatoires + `.gitignore`

Éléments confirmés non vulnérables

- **Injection SQL** : SQLMap — 4 144 requêtes sans succès
- **Bypass JWT/RBAC** : Modification payload détectée et rejetée
- **Manipulation audit logs** : Endpoints DELETE/PUT/POST → 404/405
- **DoS upload** : Fichier 50 Mo → HTTP 413

6 Architecture du Chiffrement — Tâche Principale

Objectif assigné
Garantir que le contenu des rapports d'expertise soit chiffré au repos et inaccessible sans la clé de déchiffrement appropriée.

6.1 Architecture implémentée

1. Algorithme

AES-256 via `AES_ENCRYPT()` MySQL. Clé dérivée par `SHA2(secret, 256)`.

2. Table de stockage

```sql
CREATE TABLE reports (
    report_id INT AUTO_INCREMENT PRIMARY KEY,
    encrypted_content LONGBLOB,  -- AES-256 chiffre
    ...
);
```

3. Flux complet de sécurité

Étape	Description
1	Inspecteur authentifié → POST /api/reports/
2	RBAC vérifie INSPECTOR ou ADMIN
3	Python lit AES_REPORT_KEY depuis `.env`
4	Clé transmise à MySQL en paramètre préparé
5	Procédure construit et encrypte le rapport
6	Trigger enregistre l'action dans `audit_logs`
7	Rapport chiffré retourné en base64
8	Déchiffrement pour export PDF avec même clé

6.2 Faille critique détectée

La clé AES était codée en clair dans le fichier SQL (App without AI) ou totalement absente (App with AI)
Cette faille annulait toute la valeur du chiffrement.

7 Évaluation des Risques

7.1 App with AI — Scores CVSS

ID	Sévérité	CVSS	Impact Métier
F-01	CRITIQUE	9,8	Contournement auth complet
F-02	CRITIQUE	9,1	Données structurelles exposées
F-03	ÉLEVÉ	8,1	Prise de contrôle de compte
F-04	ÉLEVÉ	7,5	Force brute sur tous les comptes
F-05	ÉLEVÉ	7,4	Exfiltration cross-origin
F-06	MOYEN	6,5	Accès non autorisé 7 jours
F-07	MOYEN	5,3	Corruption intégrité BDD
F-08	MOYEN	7,2	Exécution code distant
F-09	MOYEN	5,3	Aucune détection de violation
F-10	FAIBLE	5,9	BDD sans authentification

Score composite avant : 73/100 (RISQUE ÉLEVÉ)
Score composite après : 18/100 (RISQUE FAIBLE)

7.2 App without AI — Synthèse

ID	Faille	Sévérité	Statut
V-01	Clé AES dans SQL	CRITIQUE	✓Corrigé
V-02	Clé AES dans `reports.py`	CRITIQUE	✓Corrigé
V-03	XSS stocké	ÉLEVÉ	✓Corrigé
V-04	Rate limiting absent	ÉLEVÉ	✓Corrigé
V-05	Erreur MySQL exposée	ÉLEVÉ	✓Corrigé
V-06	Procédures absentes	ÉLEVÉ	✓Corrigé
V-07	Secrets placeholder	CRITIQUE	✓Corrigé

Score après correction : 15/100 (RISQUE FAIBLE)

8 Comparaison des Deux Applications

8.1 Tableau comparatif

Critère	App with AI	App without AI
Vulnérabilités critiques	2	3
Vulnérabilités élevées	3	4
Vulnérabilités moyennes	4	0
Vulnérabilités faibles	1	0
Score risque initial	73/100 (ÉLEVÉ)	85/100 (ÉLEVÉ)
Score risque final	18/100 (FAIBLE)	15/100 (FAIBLE)
Chiffrement initial	Absent	Présent (clé hardcodée)
Rate limiting initial	Absent	Absent
2FA	Ajoutée	Absente
Journalisation audit	Ajoutée	Partielle
Validation entrées	Modérée	Robuste (Pydantic)
Architecture	Flask	FastAPI

8.2 Analyse qualitative

App with AI

Forces :

- Scripts de migration complets
- 2FA implémentée
- Journalisation d'audit complète

Faiblesses :

- Plus de vulnérabilités (10)
- `debug=True`, CORS wildcard
- TTL JWT 7 jours

App without AI

Forces :

- Validation Pydantic robuste
- Architecture FastAPI moderne
- RBAC bien implémenté

Faiblesses :

- Clé AES hardcodée (critique)
- Absence rate limiting
- Erreurs MySQL exposées

8.3 Conclusion générale

L'application générée par IA présentait un spectre plus large de vulnérabilités (10 vs 7),
mais avec des failles critiques moins nombreuses.

L'application sans IA avait moins de vulnérabilités mais des failles plus critiques sur le chiffrement.

Recommandation finale

1. Conserver l'architecture FastAPI (App without AI) pour sa robustesse native
2. Intégrer les correctifs de l'App with AI : 2FA, journalisation, rate limiting
3. Maintenir une rotation régulière des clés de chiffrement
4. Commander un test d'intrusion dans les 30 jours

9 Annexes

9.1 Annexe A — Scripts de Migration SQL

AUDIT_LOG

```sql
CREATE TABLE IF NOT EXISTS AUDIT_LOG (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    timestamp DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
    user_id INT NULL,
    ip_address VARCHAR(45) NOT NULL,
    action VARCHAR(100) NOT NULL,
    success BOOLEAN NOT NULL,
    FOREIGN KEY (user_id) REFERENCES UTILISATEUR(id_utilisateur)
);
```

Colonnes 2FA

```sql
ALTER TABLE UTILISATEUR
    ADD COLUMN IF NOT EXISTS totp_secret VARCHAR(128) NULL,
    ADD COLUMN IF NOT EXISTS totp_enabled BOOLEAN NOT NULL
   DEFAULT FALSE;
```

9.2 Annexe B — Variables d'Environnement

Variable	Requis	Description
JWT_SECRET	Oui	Hex aléatoire 256 bits
FIELD_ENCRYPTION_KEY	Oui	Clé AES-256-GCM
AES_REPORT_KEY	Oui	Clé AES pour rapports
DB_PASSWORD	Oui (prod)	Mot de passe MySQL
FLASK_ENV / APP_ENV	Oui	'production' en prod

9.3 Annexe C — Glossaire

Terme	Définition
AES-256-GCM	Chiffrement authentifié — NIST SP 800-38D
CORS	Contrôle des origines cross-origin
CVSS v3.1	Système de notation des vulnérabilités (0-10)
JWT	Jeton signé pour authentification sans état
TOTP	Mot de passe à usage unique basé sur le temps (RFC 6238)
2FA / MFA	Authentification à deux facteurs
RBAC	Contrôle d'accès basé sur les rôles
STRIDE	Modèle de menaces (Usurpation, Falsification, Répudiation, Divulgation, DoS, Élévation)